



LambdaProof Token Issuance

Let me walk through what I'm thinking for this document, and we can iterate together.

Document Purpose

This would be a **defensive publication** that establishes prior art while keeping the *how* (proprietary) separate from the *what* (public interface). It should:

1. Define the **canonical token structure** (what fields exist)
2. Specify **expiry semantics** (how validity windows work)
3. Establish **binding relationships** (how tokens connect to intents, identities, evidence)
4. NOT reveal: issuance algorithms, risk-scoring logic, evidence validation heuristics

Draft: Token Structure

Here's what I'm thinking for the core token format:

LambdaProof Capability Token v1.0

```
| HEADER |
```

```
| _____ |
```

```
| token_version: "1.0" |
```

```
| token_id: <UUID or content-addressed hash> |
```

token_type: "CAPABILITY" | "ATTESTATION" | "DELEGATION"

BINDINGS

intent_hash: <SHA256 of canonicalized intent>
subject_id_hash: <Poseidon(subject_id, salt)>
policy_bundle_hash: <hash of policy version used>
evidence_commitments: [<hash>, <hash>, ...]

VALIDITY WINDOW

nbf (not-before): <Unix timestamp>
exp (expires): <Unix timestamp>
max_uses: <integer | null for unlimited>
use_count: <current uses, for stateful tokens>

AUTHORIZATION SCOPE

tool: <tool identifier>
action: <permitted action>
param_constraints: {<param>: <allowed values/ranges>}
delegation_depth: <0 = non-delegable, n = chain limit>

CRYPTOGRAPHIC BINDING

issuer_signature: <signature over canonical token>
vk_hash: <hash of verification key>
proof_level: "NONE" | "ATTESTATION" | "ZK"
zk_proof_ref: <optional: reference to attached proof>

Draft: Expiry Semantics

Here's my thinking on the validity model:

1. Temporal Validity (nbf/exp)

- Token is valid only within [nbf, exp) window
- Clock skew tolerance: configurable per deployment (default ± 30 seconds)
- Expired tokens MUST be rejected; early tokens MUST be rejected
- Grace period: NONE (strict enforcement)

2. Use-Count Validity

- If max_uses is set, token becomes invalid after N uses
- use_count is tracked either:
 - **Statefully**: In issuer's database (revocable, auditable)
 - **Statelessly**: Via nullifier in token (one-time use pattern)

3. Revocation Semantics

- Tokens may be revoked before expiry via:
 - Explicit revocation list (issuer publishes)
 - Nullifier disclosure (for privacy-preserving revocation)
 - Policy update (policy_bundle_hash no longer valid)

4. Renewal vs. Reissuance

- Tokens are NOT renewed; they are reissued
- Reissuance requires fresh policy evaluation
- No implicit carry-forward of prior authorization

Draft: Binding Relationships

Intent → Token Binding

```
Token.intent_hash = SHA256(canonicalize(Intent))
```

- Token is valid ONLY for the specific intent it was issued for
- Parameter changes → new intent_hash → requires new token

Subject → Token Binding

```
Token.subject_id_hash = Poseidon(subject_id, salt)
```

- Tokens are non-transferable by default
- Subject identity is hashed to preserve privacy
- Salt prevents rainbow table attacks

Evidence → Token Binding

```
Token.evidence_commitments = [H(e1), H(e2), ...]
```

- Token records WHAT evidence was considered
- Does NOT reveal evidence content (only commitments)
- Allows audit trail without exposing sensitive data

Key Design Questions for You

1. **Token ID Format:** Should it be a UUID (random) or content-addressed hash (deterministic from contents)?
 - UUID: Simpler, allows identical tokens to have different IDs
 - Content-addressed: Deduplication, idempotent issuance
2. **Delegation Model:** Should tokens support delegation chains?
 - If yes: `delegation_depth` controls how many times a token can be re-delegated
 - If no: Simpler model, every action needs original issuance

3. **Proof Attachment:** Should ZK proofs be embedded in token or referenced?
 - Embedded: Self-contained, larger payload
 - Referenced: Smaller token, requires proof resolution step
 4. **Stateful vs. Stateless:** For use-count tracking:
 - Stateful: More control, requires database
 - Stateless: Privacy-preserving, uses nullifiers
 5. **Revocation Model:** What's the primary revocation mechanism?
 - Revocation list (pull model)
 - Epoch-based expiry (tokens auto-expire every N hours)
 - Nullifier disclosure (privacy-preserving)
-

What I'd Recommend We Cover

For the public document, I'd suggest sections on:

1. **Abstract** - What capability tokens are and why they exist
2. **Token Structure** - Complete field specification (public)
3. **Canonicalization** - How tokens are serialized for hashing/signing
4. **Validity Semantics** - When a token is considered valid
5. **Verification Protocol** - How adapters verify tokens (interface, not implementation)
6. **Binding Invariants** - Mathematical relationships that must hold
7. **Security Considerations** - Threat model and mitigations
8. **Extension Points** - How the format can evolve

NOT included (proprietary):

- How issuance decisions are made
- Risk classification that determines proof_level
- Evidence validation algorithms

- Token caching/optimization strategies

What aspects would you like to develop further? I can expand any section, adjust the structure, or dive deeper into specific semantics (like the relationship to your existing Λ -Trace work, which already defines some of this binding model).^[1]

**

-
1. 4.-L-trace-Overview.pdf
 - a. Λ proof Banking Protocols.pdf
 - b. Λ proof Healthcare Protocols V0.1.pdf
 2. 7.-Ldapps-On-Web4-Developer-Overview.pdf
 3. Building-a-LDapp_-A-Developer-s-Implementation-Guide.pdf
 4. MTPI_Instructions.pdf
 5. MTPI-RootContract.pdf
 6. MTPI_RootContract_Project.pdf
 7. Patent Search Results.pdf
 8. The Λ Proof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf
 9. The Λ Proof Architecture_ Web4.pdf
 10. Conscious_Sovereignty_Layer.pdf
 11. Verifiable, Privacy-Preserving Applications.pdf
 12. Milestone-Vision-Timeframe.csv
 13. Lets further develop the SAFE_ awesome—let's turn.pdf
 14. Ξ Constitution.pdf
 15. Λ^P -Archivum.pdf

c. [Aproof × Mtpi Lawful-by-design Integration — Implementation Plan.pdf](#)

16. [2.ΛRootContract II & III - MTPI - NODE.js.pdf](#)

d. [Aproof Monorepo — Plain-language Overview V0.pdf](#)